

N+1 Query Problem

The **N+1 Query Problem** is a common performance issue in databases and ORMs (such as Hibernate, JPA, Entity Framework, Django ORM, etc.).

It occurs when:

1. One query is executed to fetch a list of records (**1 query**).
2. Then an additional query is executed for each record (**N queries**).

As a result, the application executes **N + 1 total queries**, leading to poor performance.

Example

Consider two tables:

Users

id	name
1	Alice
2	Bob
3	Charlie

Orders

id	user_id	product
1	1	Laptop
2	1	Mouse
3	2	Keyboard

Suppose we want to display:

None

Alice -> Orders

Bob -> Orders

Charlie -> Orders

Problematic Approach

Query 1

SQL

```
SELECT * FROM users;
```

Returns:

None

Alice

Bob

Charlie

Now for each user:

Query 2

SQL

```
SELECT * FROM orders WHERE user_id = 1;
```

Query 3

SQL

```
SELECT * FROM orders WHERE user_id = 2;
```

Query 4

SQL

```
SELECT * FROM orders WHERE user_id = 3;
```

Total queries:

None

`1 + 3 = 4 queries`

If there are 10,000 users:

None

`1 + 10,000 queries`

This becomes extremely slow.

Why It Happens

The N+1 problem usually occurs because of:

Lazy Loading

Example in JPA/Hibernate:

Java

```
List<User> users = userRepository.findAll();

for(User user : users){
    System.out.println(user.getOrders());
}
```

`findAll()` loads users only.

When `getOrders()` is called, Hibernate executes another query for each user.

Performance Impact

Assume:

- 1000 users
- 5 ms per database query

Total time:

None

$1001 \times 5 \text{ ms}$

$= 5005 \text{ ms}$

$\approx 5 \text{ seconds}$

Instead of one optimized query, the application spends seconds waiting for the database.

Solutions

1. JOIN Fetch (Most Common)

Instead of multiple queries:

SQL

```
SELECT *  
FROM users u  
LEFT JOIN orders o  
ON u.id = o.user_id;
```

Only one query is executed.

Hibernate Example

Java

```
@Query(""  
SELECT u  
FROM User u
```

```
JOIN FETCH u.orders  
""")  
List<User> findAllUsersWithOrders();
```

This loads users and orders together.

2. Eager Loading

```
Java  
@OneToMany(fetch = FetchType.EAGER)  
private List<Order> orders;
```

Orders are fetched automatically.

Drawback

May load unnecessary data.

Therefore, EAGER loading should be used carefully.

3. Batch Fetching

Instead of:

```
SQL  
SELECT * FROM orders WHERE user_id = 1;  
SELECT * FROM orders WHERE user_id = 2;  
SELECT * FROM orders WHERE user_id = 3;
```

Use:

```
SQL  
SELECT *
```

```
FROM orders
WHERE user_id IN (1,2,3);
```

Hibernate example:

```
Java
@BatchSize(size = 100)
```

This significantly reduces query count.

4. DTO Projection

Fetch only required data:

```
Java
SELECT new UserOrderDTO(
    u.name,
    o.product
)
FROM User u
JOIN u.orders o
```

Benefits:

- Less memory usage
- Fewer database round trips
- Faster queries

Visual Representation

N+1 Problem

None

Application

```
|
|---- Query Users
|
|---- Query Orders(User1)
|
|---- Query Orders(User2)
|
|---- Query Orders(User3)
|
|---- Query Orders(UserN)
```

Total:

None

1 + N Queries

Optimized Version

None

Application

```
|
|---- JOIN Users + Orders
```

Total:

None

1 Query

System Design Perspective

In large-scale systems, N+1 queries can:

- Increase database load
- Increase API latency
- Cause connection pool exhaustion
- Reduce throughput
- Increase cloud costs

During system design interviews, common mitigations include:

- JOINS
- Batch fetching
- Caching (Redis)
- Denormalization
- Precomputed views
- GraphQL DataLoader pattern

Interview Definition

The N+1 Query Problem occurs when an application executes one query to fetch a list of entities and then executes an additional query for each entity to fetch related data, resulting in N+1 database queries instead of a single optimized query. This causes unnecessary database round trips and performance degradation.